

20 NumPy Interview Questions, Free Sample

NumPy

Every question was generated from real code, run for real, and checked against the actual output before being kept. Every wrong answer is explained too, not just the right one.

How this book was built

Most interview question banks are written by hand and quietly carry the mistakes of whoever wrote them. This one was built the other way around. Every code snippet in this book was run for real, and the printed answer key is the printed output, not someone's memory of what it should be. Where two independent ways of running the same snippet disagreed, the question was dropped rather than guessed at.

Each question also explains why the three wrong answers are wrong, not only why the right one is right. That is the part most practice tests skip, and it is the part that actually teaches you something the second time you get a question wrong.

NumPy Questions

Block 1, Arrays, dtypes and Scalars

1. What does this print?

```
import numpy as np
a = np.array([1, 2.5, True])
print(a.dtype)
```

- A. int64
- B. float64
- C. bool
- D. object

2. What does this print?

```
import numpy as np
a = [np.int64(1), np.int64(2)]
print(a)
```

- A. [np.int64(1), np.int64(2)]
- B. ['np.int64(1)', 'np.int64(2)']
- C. [int64(1), int64(2)]
- D. [1, 2]

3. What does this print?

```
import numpy as np
a = np.array([1, 2, 3], dtype=np.float32)
b = a + 1.5
print(b.dtype)
```

- A. int64
- B. float32
- C. float64
- D. object

Block 2, Indexing, Slicing and Masks

4. What does this print?

```
import numpy as np
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(m[1, 2])
```

- A. 4
- B. np.int64(6)
- C. 6
- D. 8

5. What does this print?

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
fancy = a[[0, 1, 2]]
fancy[0] = 999
print(a.tolist())
```

- A. [999, 20, 30]
- B. [10, 20, 30]
- C. [10, 20, 30, 40, 50]
- D. [999, 20, 30, 40, 50]

6. What does this print?

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
a[[0, 1]] = [111, 222]
print(a.tolist())
```

- A. [111, 222, 30, 40, 50]
- B. [222, 111, 30, 40, 50]
- C. [10, 20, 30, 40, 50]
- D. [111, 222, 30]

Block 3, Broadcasting and Shapes

7. What does this print?

```
import numpy as np
a = np.arange(3)[: , np.newaxis]
b = np.arange(4)[np.newaxis, :]
print((a + b).tolist())
```

- A. [[0, 1, 2, 3], [0, 1, 2, 3], [0, 1, 2, 3]]
- B. [[0, 1, 2, 3], [1, 2, 3, 4], [2, 3, 4, 5]]
- C. [[0, 0, 0, 0], [1, 1, 1, 1], [2, 2, 2, 2]]
- D. [[0, 1, 2], [1, 2, 3], [2, 3, 4], [3, 4, 5]]

8. What does this print?

```
import numpy as np
a = np.arange(3)
print(a[np.newaxis, :].shape)
```

- A. (3, 1)
- B. (1, 1, 3)
- C. (1, 3)
- D. (3,)

Block 4, Views, Copies and Reshape

9. What does this print?

```
import numpy as np
a = np.arange(6)
b = a[[1, 2, 3]]
print(b.base is None)
```

- A. False
- B. True
- C. AttributeError
- D. None

10. What does this print?

```
import numpy as np
a = np.arange(6)
print(a.reshape(4, -1).shape)
```

- A. (4, 2)
- B. (4, 1)
- C. ValueError
- D. (4, 6)

Block 5, Reductions and the axis Argument

11. What does this print?

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.sum(axis=1).tolist())
```

- A. [6, 15]
- B. 21
- C. [3, 6]
- D. [5, 7, 9]

12. What does this print?

```
import numpy as np
h = np.array([[1, 2], [3, 4]])
print(h.cumsum(axis=1).tolist())
```

- A. [[1, 2], [4, 6]]
- B. [1, 3, 6, 10]
- C. [[1, 2], [3, 4]]
- D. [[1, 3], [3, 7]]

13. What does this print?

```
import numpy as np
b = np.array([[1, 2], [3, 4], [5, 6]])
print(b.argmax())
```

- A. 5
- B. 6
- C. (2, 1)

D. 1

Block 6, NaN, Overflow and Type Promotion

14. What does this print?

```
import numpy as np
a = np.array([1.0, np.nan, 3.0])
print(a.max())
```

- A. nan
- B. ValueError
- C. 3.0
- D. 1.0

15. What does this print?

```
import numpy as np
a = np.array([1, 2], dtype=np.int8)
print((a + 127).tolist())
```

- A. OverflowError
- B. [126, 127]
- C. [-128, -127]
- D. [128, 129]

Block 7, Sorting, Searching and unique

16. What does this print?

```
import numpy as np
a = np.array([3, 1, 2])
r = a.sort()
print(r, a.tolist())
```

- A. None [1, 2, 3]
- B. None [3, 1, 2]
- C. [1, 2, 3] [1, 2, 3]
- D. array([1, 2, 3]) [1, 2, 3]

17. What does this print?

```
import numpy as np
a = np.array([[3, 1], [2, 4]])
print(np.sort(a, axis=1).tolist())
```

- A. [[1, 3], [2, 4]]
- B. [[2, 1], [3, 4]]
- C. [[3, 1], [4, 2]]
- D. [[1, 2], [3, 4]]

18. What does this print?

```
import numpy as np
a = np.array([[1, 3], [2, 0]])
print(np.argsort(a, axis=1).tolist())
```

- A. [[1, 0], [1, 0]]
- B. [[0, 1], [1, 0]]
- C. [[0, 1], [0, 1]]
- D. [[1, 0], [0, 1]]

Block 8, Stacking, Splitting and Concatenation

19. What does this print?

```
import numpy as np
a = np.arange(9)
print(np.split(a, 3))
```

- A. [array([0, 1, 2]), array([3, 4, 5])]
- B. [array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
- C. [array([0, 3, 6]), array([1, 4, 7]), array([2, 5, 8])]
- D. [array([0, 1, 2, 3, 4, 5, 6, 7, 8])]

20. What does this print?

```
import numpy as np
a = np.array([1, 2])
b = np.array([3, 4, 5])
print(np.concatenate([a, b]).tolist())
```

- A. ValueError
- B. [3, 4, 5, 1, 2]
- C. [1, 2, 3, 4]
- D. [1, 2, 3, 4, 5]

Answers and Explanations

Organized in the same order as the questions. Each option is explained, the correct one and the three distractors.

NumPy Answers

1. Correct answer, **B**

A. `int64`

Assumes the two non float values outvote the single float and the dtype settles on `int64`.

B. (correct) `float64`

Correct. With `bool`, `int` and `float` all present, NumPy still promotes to the widest of the three, `float64`.

C. `bool`

Assumes the presence of a boolean anywhere forces the narrower `bool` dtype regardless of the other values.

D. `object`

Assumes a three way mix of value kinds is too ambiguous for a numeric dtype and falls back to `object`.

The promotion rule does not count votes, it only asks which dtype among those present can hold every value without loss. With `int`, `float` and `bool` together, `float64` is that dtype, exactly as it would be with only two of the three kinds present.

2. Correct answer, **A**

A. (correct) `[np.int64(1), np.int64(2)]`

Correct. Printing a plain Python list calls `repr` on each element, so the scalars show their NumPy 2 type wrapper even though `print` itself uses `str` on the outer list.

B. `['np.int64(1)', 'np.int64(2)']`

Assumes the wrapped `repr`s come out as quoted text rather than as the literal `repr` NumPy produces.

C. `[int64(1), int64(2)]`

Assumes the wrapper drops the `np` module prefix inside a list the way it never does for a bare scalar either.

D. `[1, 2]`

Assumes the elements print the same bare way they would if printed on their own with `print`.

A Python list has no special display rule of its own, it falls back to calling `repr` on every element it contains. Since NumPy 2 scalar `repr` includes the type name, elements inside a list show that wrapper even when printing the same scalar alone would only show the bare value.

3. Correct answer, **B**

A. `int64`

Assumes mixing a `float32` array with a Python float drops back to a whole number dtype.

B. (correct) `float32`

Correct. The same NEP 50 rule applies to floats, a Python float added to a `float32` array keeps the result `float32` rather than upgrading it to `float64`.

C. `float64`

Assumes the bare Python float on the right widens the result to the default double precision dtype, the way older NumPy versions behaved.

D. `object`

Assumes this kind of mixed operation needs a generic `object` dtype to represent the result.

A Python scalar in an arithmetic operation no longer forces the array to a wider dtype based on its own kind, it only has to fit within the dtype the array already has. A float32 array plus a Python float stays float32 under this rule.

4. Correct answer, C

A. 4

Reads the indices as column first and row second, landing on the value at the start of row 1 instead.

B. `np.int64(6)`

Confuses printing a lone scalar with printing it inside a list, which would show the type wrapper.

C. (correct) 6

Correct. The comma form selects row 1 and column 2 in one step, landing on 6.

D. 8

Reads row 2 and column 1 instead of row 1 and column 2.

*`a[i, j]` on a 2D array selects row *i* and column *j* directly with one indexing operation. Row 1 here is 4, 5, 6, and column 2 of that row is 6.*

5. Correct answer, C

A. [999, 20, 30]

Assumes the edit does reach the original array but only up to the length of the fancy selection.

B. [10, 20, 30]

Assumes the entire original array was replaced by the shorter fancy selection.

C. (correct) [10, 20, 30, 40, 50]

Correct. Fancy indexing always returns a new array, so changing fancy afterward has no effect on the original array `a`, which stays 10, 20, 30, 40, 50.

D. [999, 20, 30, 40, 50]

Assumes fancy indexing returns a view into the same memory, so the edit reaches back into the original array.

Fancy indexing with a list of positions always copies the selected elements into a brand new array, it never shares memory with the source. This is the key difference from a slice, which returns a view, editing a fancy indexed result never changes the array it was built from.

6. Correct answer, A

A. (correct) [111, 222, 30, 40, 50]

Correct. Assigning through a fancy index writes directly into the original array at the named positions, position 0 becomes 111 and position 1 becomes 222.

B. [222, 111, 30, 40, 50]

Swaps which value lands on which position, as if the assigned list were applied in reverse order.

C. [10, 20, 30, 40, 50]

Assumes an assignment through a fancy index list has no effect on the original array, the way reading through one produces a copy.

D. [111, 222, 30]

Assumes the assignment also shortens the array down to only the two positions that were written.

Fancy indexing produces a copy when reading, but assigning through the same kind of index writes straight into the original array at each named position. This is a genuine exception to the reading rule and is exactly how a batch update to specific positions is usually written.

7. Correct answer, B

A. `[[0, 1, 2, 3], [0, 1, 2, 3], [0, 1, 2, 3]]`

Ignores the contribution of a entirely and repeats b unchanged on every row.

B. (correct) `[[0, 1, 2, 3], [1, 2, 3, 4], [2, 3, 4, 5]]`

Correct. Each row i adds the constant a value i to every entry of b, so row 0 is b itself, row 1 is b plus 1, and row 2 is b plus 2.

C. `[[0, 0, 0, 0], [1, 1, 1, 1], [2, 2, 2, 2]]`

Ignores the contribution of b entirely and repeats each value of a across a whole row.

D. `[[0, 1, 2], [1, 2, 3], [2, 3, 4], [3, 4, 5]]`

Transposes the grid, producing 4 rows of length 3 instead of 3 rows of length 4.

Broadcasting a column against a row builds a grid where every cell is the sum of that row's value from the column operand and that column's value from the row operand. Neither operand ever gets dropped from the result, both contribute to every single cell.

8. Correct answer, C

A. (3, 1)

Places the new axis after the existing one instead of before it.

B. (1, 1, 3)

Assumes np.newaxis inserts two new axes instead of exactly one.

C. (correct) (1, 3)

Correct. np.newaxis inserts a size 1 axis exactly where it appears in the index, before the existing axis, turning shape (3,) into (1, 3).

D. (3,)

Assumes indexing with np.newaxis alongside a full slice leaves the shape completely unchanged.

np.newaxis inserts exactly one size 1 axis at the position it occupies in the index tuple, everything else about the array stays the same. Putting it before the colon makes a row shape (1, 3), putting it after would make a column shape (3, 1) instead.

9. Correct answer, B

A. False

Assumes any array built from indexing another array must carry a base pointing back to it.

B. (correct) True

Correct. Fancy indexing produces an independent array that owns its own memory, so it has no source array to point back to and its base is None.

C. AttributeError

Assumes an array produced by indexing has no base attribute to check at all.

D. None

Assumes base reports the missing marker None as a printed value instead of the array being asked whether its base is that value.

The base attribute distinguishes an array that owns its memory from one that only views someone else's. Because fancy indexing always copies, the result owns its own freshly allocated buffer and its base is None, exactly like an array built directly with a constructor.

10. Correct answer, C

A. (4, 2)

Assumes -1 rounds up to the nearest whole number of columns instead of requiring an exact division.

B. (4, 1)

Assumes a single column is produced regardless of whether the element count actually divides evenly by the fixed axis.

C. (correct) ValueError

Correct. 6 elements split across 4 rows would need 1.5 columns, which is not a whole number, so NumPy raises ValueError instead of producing a shape that could not hold every element exactly once.

D. (4, 6)

Assumes reshape repeats some elements to fill out an inexact shape.

Just like reshape(-1, 4), fixing one axis and asking for -1 on the other only succeeds when the total element count divides evenly by the fixed size. Four does not divide six evenly, so there is no valid whole number for the other axis and the call fails.

11. Correct answer, A

A. (correct) [6, 15]

Correct. axis=1 collapses the columns and adds across each row, giving 1+2+3 for the first row and 4+5+6 for the second.

B. 21

Collapses the whole array into one grand total instead of one sum per row.

C. [3, 6]

Keeps only the last entry of each row rather than adding the whole row.

D. [5, 7, 9]

Sums down each column instead, the result for axis=0 on the same array.

Reducing along axis=1 keeps one result per row and drops the column axis entirely. It is easy to swap axis=0 and axis=1 in your head, the safest check is to look at how many numbers come out, one per row means axis=1.

12. Correct answer, D

A. [[1, 2], [4, 6]]

Accumulates down each column instead, the result for axis=0 on the same array.

B. [1, 3, 6, 10]

Flattens the array first, the result cumsum would give without any axis at all.

C. [[1, 2], [3, 4]]

Assumes cumsum leaves the array unchanged when an axis is given.

D. (correct) [[1, 3], [3, 7]]

Correct. axis=1 accumulates across each row, so the first row becomes 1 and 1+2, and the second row becomes 3 and 3+4.

cumsum(axis=1) resets its running total at the start of every row and walks left to right within that row only. Rows never influence each other under this axis, unlike axis=0 where columns accumulate independently of each other.

13. Correct answer, A

A. (correct) 5

Correct. Flattened in row major order the values are 1, 2, 3, 4, 5, 6, and the largest one, 6, sits at the last position, index 5.

B. 6

Confuses the index of the maximum with the maximum value itself.

C. (2, 1)

Assumes argmax returns the row and column coordinates as a tuple instead of a single flat index.

D. 1

Confuses the index of the maximum with the row it happens to sit in.

argmax without an axis always flattens the array first, so its shape does not matter, only how many elements it has and their order. The returned index is a position into that flattened sequence, not a pair of coordinates.

14. Correct answer, A

A. (correct) nan

Correct. Ordinary max() does not skip NaN, so its presence anywhere in the array poisons the whole comparison and the result is NaN.

B. ValueError

Assumes comparing against a NaN is invalid and must raise an error.

C. 3.0

Assumes max() quietly ignores the NaN and reports the largest real value instead.

D. 1.0

Assumes max() reports the smallest real value rather than propagating the NaN.

Just like sum() and mean(), the plain max() reduction propagates a NaN rather than skipping it, since any comparison against NaN is neither true nor false in a useful sense. np.nanmax exists specifically to ignore NaN entries when they should not count.

15. Correct answer, C

A. OverflowError

Assumes the Python int 127 does not fit an int8 array and must raise before any arithmetic happens.

B. [126, 127]

Subtracts instead of adds, as if the operator had been reversed.

C. (correct) [-128, -127]

Correct. 127 fits within int8 so the addition proceeds, but the true sums, 128 and 129, both overflow int8 and wrap to negative 128 and negative 127.

D. [128, 129]

Assumes the addition returns the true mathematical sums without wrapping them into the int8 range.

127 is itself a valid int8 value, so NEP 50 lets the addition proceed using the array dtype, it is only the arithmetic result that overflows and wraps. This is the same distinction as with uint8, an in range Python operand can still produce an out of range, silently wrapped result.

16. Correct answer, A

A. (correct) None [1, 2, 3]

Correct. The in place method a.sort() reorders the array itself and returns None, so printing its result shows None while a itself now holds the sorted values.

B. None [3, 1, 2]

Assumes a.sort() returns None without actually reordering the array in place.

C. [1, 2, 3] [1, 2, 3]

Assumes a.sort() returns the sorted array the way np.sort would, instead of returning None.

D. array([1, 2, 3]) [1, 2, 3]

Assumes the in place method hands back a printed array object rather than the plain value None.

a.sort() is an in place method, it reorders the array itself and returns None, exactly like list.sort() in plain Python. Only the free function np.sort returns a new sorted array that can be captured in a variable.

17. Correct answer, A

A. (correct) `[[1, 3], [2, 4]]`

Correct. `axis=1` sorts each row independently across its columns, so 3, 1 becomes 1, 3 and 2, 4 stays 2, 4.

B. `[[2, 1], [3, 4]]`

Sorts along the columns instead of the rows, the result `axis=0` would actually give.

C. `[[3, 1], [4, 2]]`

Sorts each row in descending order instead of the default ascending order.

D. `[[1, 2], [3, 4]]`

Assumes the whole array gets flattened, sorted, and reshaped back, ignoring the axis argument entirely.

`axis=1` on a 2D array sorts independently within each row, rearranging only the values that share a row. The default axis for `np.sort` on a multi dimensional array is the last one, which for a 2D array is exactly this row wise behavior.

18. Correct answer, B

A. `[[1, 0], [1, 0]]`

Repeats the second row twice for pattern reasons, instead of noticing the first row was already in order.

B. (correct) `[[0, 1], [1, 0]]`

Correct. `argsort` with `axis=1` works row by row, and the first row is already ascending so its indices stay 0, 1, while the second row needs its two entries swapped, giving indices 1, 0.

C. `[[0, 1], [0, 1]]`

Assumes both rows produce the same index order, ignoring that the second row is not already sorted.

D. `[[1, 0], [0, 1]]`

Swaps the two rows of the result, giving the second row the indices that belong to the first.

`argsort` with an explicit axis works independently along that axis, exactly like `np.sort` does, row by row when `axis=1`. A row that is already in ascending order simply returns its original positions unchanged.

19. Correct answer, B

A. `[array([0, 1, 2]), array([3, 4, 5])]`

Drops one of the three expected pieces from the result.

B. (correct) `[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]`

Correct. `split` with an integer count of 3 cuts the nine values into three equal, contiguous chunks in order, and it returns a plain Python list of the resulting arrays.

C. `[array([0, 3, 6]), array([1, 4, 7]), array([2, 5, 8])]`

Assumes `split` distributes the values round robin across the pieces instead of keeping each chunk contiguous.

D. `[array([0, 1, 2, 3, 4, 5, 6, 7, 8])]`

Assumes `split` with a count argument returns the array untouched inside a single element list.

`np.split` with an integer divides the array into that many equal, contiguous pieces along the given axis, always returning a plain list of arrays rather than a single array. It only works when the axis length divides evenly by that integer, otherwise it raises instead of guessing a chunk size.

20. Correct answer, D

A. `ValueError`

Assumes `concatenate` requires every input to have exactly the same length, the way it requires matching sizes on the other axes for multi dimensional arrays.

B. `[3, 4, 5, 1, 2]`

Joins the two arrays in the wrong order.

C. [1, 2, 3, 4]

Drops one of the values from the longer array instead of keeping every value from both arrays.

D. (correct) [1, 2, 3, 4, 5]

Correct. For 1D arrays, concatenate only needs the arrays to share the same number of dimensions, their lengths along the join axis are free to differ, so two and three values simply add up to five.

The shape rule for concatenate only constrains the axes that are not being joined, the axis being joined is exactly the one allowed to differ in size. For plain 1D arrays there is no other axis to disagree on, so any two 1D arrays can be concatenated regardless of their individual lengths.

That was 20 of the 279 NumPy questions.

The full bank is one PDF laid out exactly like this sample, all 279 questions first and the complete answer key after, with a reason for each of the four options. Every snippet in it was run the same way before its answer was written.

Get it for 9 euros at <https://ko-fi.com/s/bc22cdcdd3>, or SQL, Python, pandas and NumPy in one PDF for 19 euros at <https://ko-fi.com/s/8151d45764>.

The same 20 questions can be played in a browser at <https://thibaudlepan77-svg.github.io/interview-questions-verified/>. If an answer looks wrong to you, open an issue on the repository linked from that page with the question number and what your own run printed.